

4-bit Digital Calculator

Table of Contents

1. Introduction	3
2. Objective	4
3. Technical Specification	4
3.1. Inputs	4
3.2. Outputs	5
3.3. Operations Performed	5
3.4. Arithmetic Unit	5
4. Design Methodology	6
4.1. 4-Bit Adder Design	6
1. Sum (S) K-Map	8
2. Carry Output (Cout)K-Map	9
4.3. 4-Bit Comparator Design	14
5. Flags	17
6. Operational Selection Mapping	17
Summary of Selection Mapping	18
7. Conclusion	19

1. Introduction

Digital systems form the foundation of modern electronic devices such as computers, calculators, mobile phones, communication systems, and embedded controllers. At the core of these systems are combinational logic circuits, which perform arithmetic and logical operations based entirely on present input values. One of the most important combinational logic components used in digital systems is the Arithmetic Logic Unit (ALU).

An ALU is responsible for performing arithmetic operations such as addition and subtraction, as well as logical and comparison operations. These operations are essential in processors, microcontrollers, and digital computation systems. Understanding the design and implementation of an ALU provides fundamental knowledge in digital logic design and computer architecture.

This project focuses on the design and simulation of a 4-bit digital calculator (4-bit ALU) using standard combinational logic gates. The system is designed to accept two 4-bit binary inputs and perform three major operations: binary addition, binary subtraction using 2's complement method, and magnitude comparison. The operation performed by the circuit is selected using control signals (S1) and (S0).

The arithmetic section of the design is implemented using a Ripple Carry Adder (RCA) structure composed of cascaded full adders. Addition is performed directly through the RCA, while subtraction is achieved using the 2's complement technique by conditionally inverting the second operand and applying an input carry of one. The comparison section is implemented using magnitude comparator logic capable of determining whether one binary number is greater than, less than, or equal to the other.

The design process involves Boolean analysis, truth table development, Karnaugh Map simplification, gate-level circuit implementation, and simulation verification. Standard logic gates such as AND, OR, NOT, XOR, and XNOR are used throughout the design in accordance with the project requirements.

Simulation and testing are performed to verify the correctness of each operation under different input conditions. The project demonstrates the practical application of combinational logic design principles and provides a strong foundation for understanding more advanced digital systems and processor architectures.

Objective

To design, analyze, and simulate a 4-bit Arithmetic Logic Unit (ALU) capable of performing binary addition, subtraction, and magnitude comparison using combinational logic circuits and standard logic gates.

1. To design a 4-bit Ripple Carry Adder (RCA) for performing binary addition.
2. To implement binary subtraction using the 2's complement method.
3. To design a 4-bit magnitude comparator capable of generating:
 - $(A > B)$
 - $(A < B)$
 - $(A = B)$
4. To develop truth tables and simplified Boolean expressions for the arithmetic and comparison circuits.
5. To apply Karnaugh Map (K-Map) techniques for Boolean function minimization and gate reduction.
6. To implement the complete system using standard logic gates including:
 - AND
 - OR
 - NOT
 - XOR
 - XNOR
7. To simulate and verify the functionality of the designed circuits using digital simulation software.
8. To analyze the operation and performance of combinational logic circuits used in digital systems.
9. To gain practical experience in digital circuit design, troubleshooting, and system integration.

3. Technical Specification

The designed system is a 4-bit Arithmetic Logic Unit (ALU) capable of performing arithmetic and comparison operations on two 4-bit binary inputs.

3.1. Inputs

The system accepts:

- Two 4-bit binary numbers:

- $A = A_3 A_2 A_1 A_0$
- $B = B_3 B_2 B_1 B_0$
- Two selection control inputs:
 - S1
 - S0

3.2. Outputs

The system produces:

- A 4-bit result output
- Carry-out output for arithmetic operations
- Comparison outputs:
 - $A > B$
 - $A < B$
 - $A = B$

3.3. Operations Performed

The operation of the ALU depends on the selection inputs S1 and S0.

S1	S0	Operation
0	0	Addition $((A + B))$
0	1	Subtraction $((A - B))$ using 2's complement
1	0	Magnitude Comparison

3.4. Arithmetic Unit

The arithmetic section is implemented using a 4-bit Ripple Carry Adder (RCA) constructed from cascaded full adders.

Subtraction Method

Subtraction is performed using the 2's complement technique:

$$A-B=A+(\bar{B}+1)$$

where the bits of B are inverted using XOR gates and the carry input is set to logic 1 during subtraction mode.

$$A-B=A+(\bar{B}+1)$$

The comparator circuit determines whether:

- $A > B$
- $A < B$
- $A = B$ using combinational logic gates.

Implementation Constraints

The design uses standard digital logic gates including:

- AND gates
- OR gates
- XOR gates
- XNOR gates
- NOT gates

The circuit is simulated using digital logic simulation software.

4. Design Methodology

4.1. 4-Bit Adder Design

The addition operation of the ALU was implemented using a 4-bit Ripple Carry Adder (RCA). The Ripple Carry Adder is constructed by cascading four full adders together. Each full adder performs the addition of two input bits together with an input carry.

The adder accepts two 4-bit binary numbers, A and B, and produces a 4-bit sum output together with a carry-out bit.

The operation of each full adder is based on the following equations:

Sum equation

$$S=A\oplus B\oplus C_{in}$$

Carry equation

$$C_{out}=AB+AC_{in}+BC_{in}$$

In the Ripple Carry Adder structure, the carry output of one stage is connected to the carry input of the next stage. Therefore, the carry signal propagates from the least significant bit (LSB) stage to the most significant bit (MSB) stage.

The first full adder receives an initial carry input of zero during normal addition mode. The resulting outputs form the binary sum of the two input numbers.

The adder circuit was implemented using basic logic gates including XOR, AND, and OR gates according to the required design constraints. Simulation results verified that the circuit correctly performs 4-bit binary addition for different input combinations.

Truth Table Explanation

The truth table is used to describe the behavior of the full adder circuit for all possible input combinations. A full adder has three inputs: A, B, and carry input (C_{in}). It produces two outputs: Sum (S) and Carry output (C_{out}).

The Sum output represents the binary addition result of the three inputs, while the Carry output indicates whether an overflow carry is generated to the next stage.

The relationship between the inputs and outputs is shown by the following equations:

Sum Output

$$S=A\oplus B\oplus C_{in}$$

Carry Output

$$C_{out}=AB+AC_{in}+BC_{in}$$

The truth table below illustrates all possible combinations of the inputs and their corresponding outputs.

A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

From the truth table, it can be observed that:

- The Sum output becomes 1 when an odd number of inputs are 1.
- The Carry output becomes 1 when at least two inputs are 1.

This truth table was used to design and verify the operation of the full adder used in the 4-bit Ripple Carry Adder structure of the ALU.

K-Map Simplification

1. Sum (S) K-Map

For the Sum output:

$$S = A \oplus B \oplus C_{in}$$

The K-Map for the Sum output is shown below.

	C _{in}	
AB	0	1
00	0	1
01	1	0
11	0	1
10	1	0

2. Carry Output (Cout)K-Map

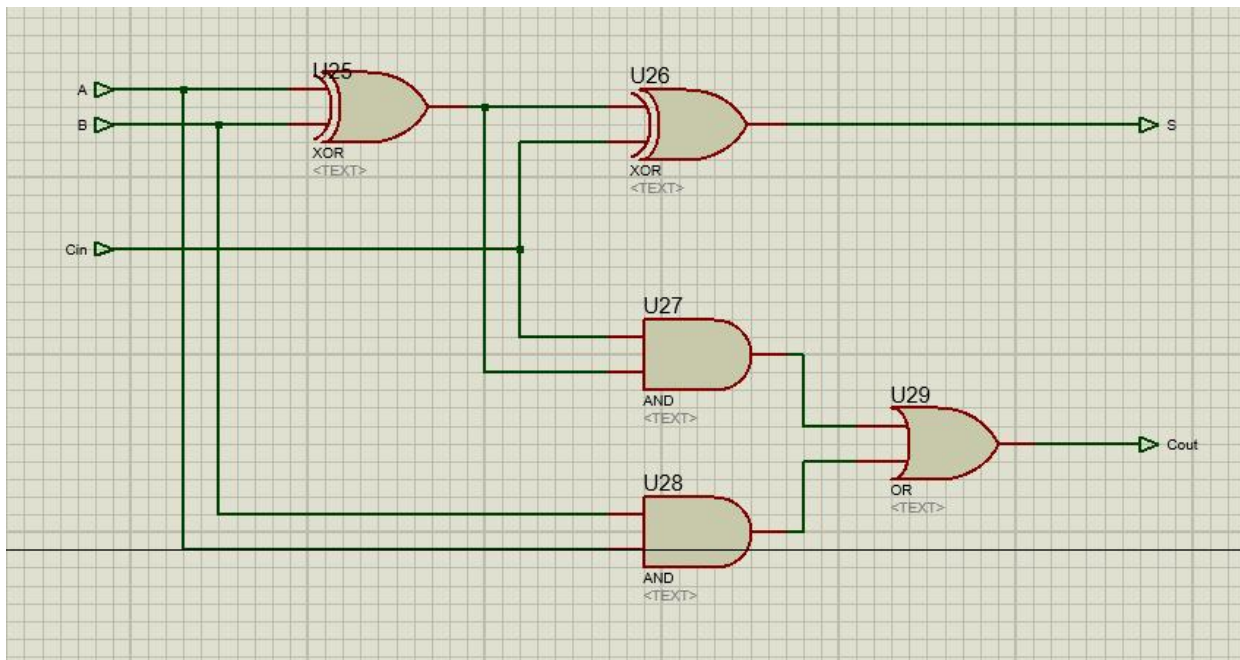
For the Carry output:

$$Cout = AB + ACin + BCin$$

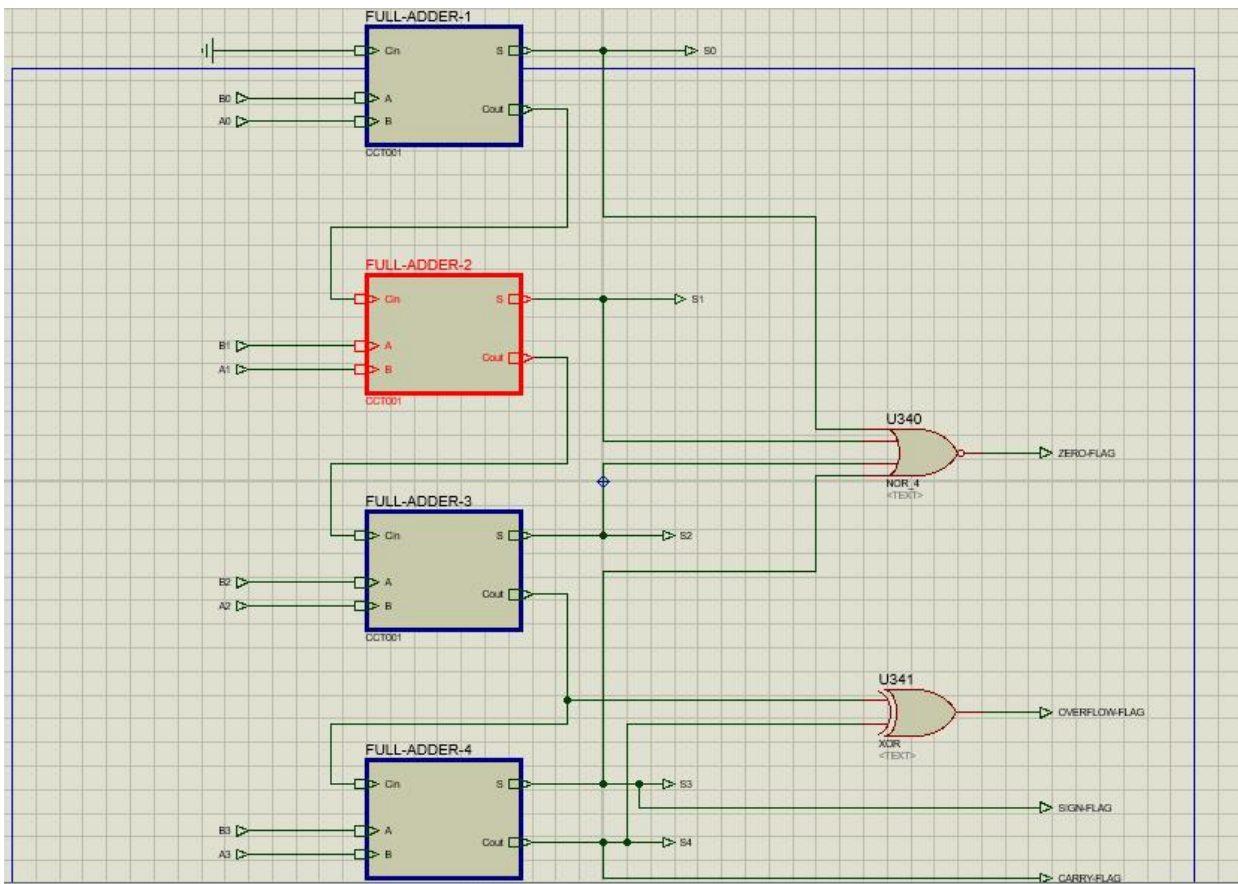
The K-Map for the Carry output is shown below.

AB \ Cin	0	1
00	0	0
01	0	1
11	1	1
10	0	1

Internal Circuit



Ripple Carry Adder Circuit



4.2. 4-Bit Subtractor Design

The subtraction operation of the ALU was implemented using a 4-bit Ripple Carry Adder (RCA) architecture. The Ripple Carry Subtractor is constructed by cascading four full adders together. To perform subtraction using 2's complement logic, each bit of the second input (B) is inverted using XOR gates, and the initial carry-in (C_{in}) of the first stage is set to 1.

The subtractor accepts two 4-bit binary numbers, A and B, and produces a 4-bit difference output together with a carry-out bit. In the Ripple Carry Adder structure used for subtraction, the carry output of one stage is connected to the carry input of the next stage. Therefore, the carry signal propagates from the least significant bit (LSB) stage to the most significant bit (MSB) stage.

The operation of each full adder within the subtractor is based on the following equations:

Sum (Difference) Equation

$$S = A \oplus B' \oplus C_{in}$$

Carry Equation

$$C_{out} = A \oplus B' + A C_{in} + B' C_{in}$$

The subtractor circuit was implemented using basic logic gates including XOR, AND, and OR gates according to the required design constraints. Simulation results verified that the circuit correctly performs 4-bit binary subtraction for different input combinations.

Truth Table Explanation

The truth table is used to describe the behavior of the full adder circuit stage when in subtraction mode. In this mode, the input to the full adder is the inverted bit of B(referred to here as B’).

A full adder has three inputs: A, B, and carry input (C_{in}). It produces two outputs: Sum (S) and Carry output (C_{out}). The Sum output represents the binary result, while the Carry output indicates the carry/borrow bit generated to the next stage.

The following table illustrates all possible combinations of the inputs and their corresponding outputs:

A	B'	C _{in}	Sum (S)	Carry (C _{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1

1	1	0	0	1
1	1	1	1	1

From the truth table, it can be observed that:

- The Difference output (S) becomes 1 when an odd number of inputs are 1.
- The Carry output becomes 1 when at least two inputs are 1.

K-Map Simplification

To ensure circuit efficiency and gate minimization, K-Maps were used to derive the simplified Boolean expressions for the subtractor stages.

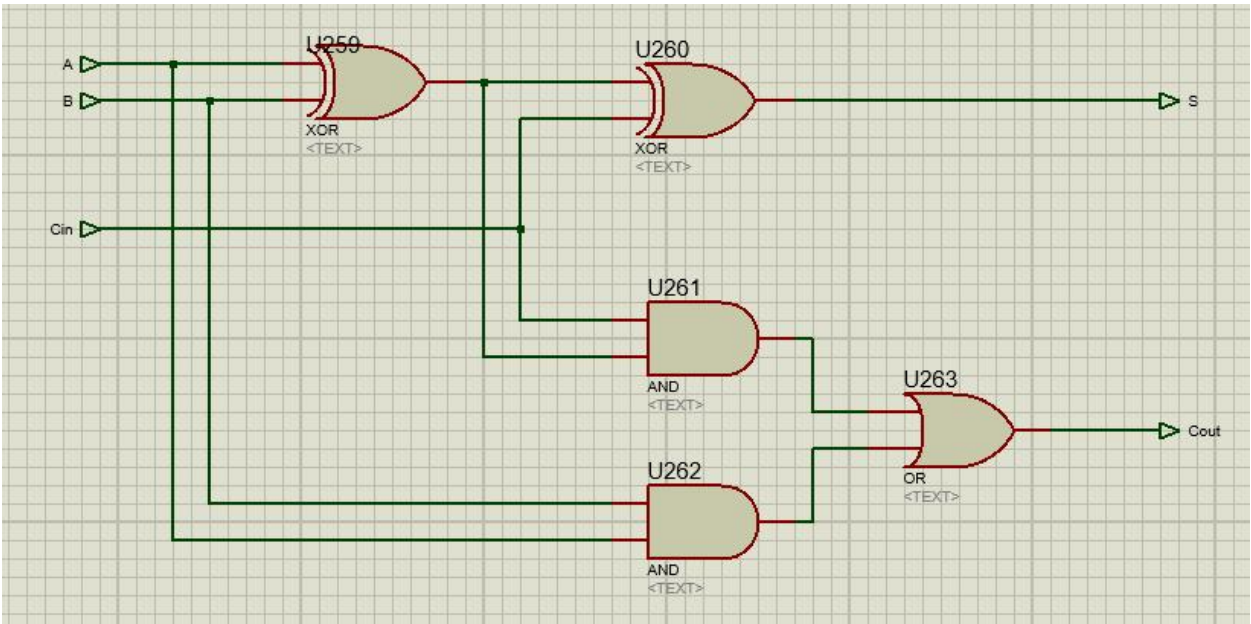
1. Difference (S) K-Map For the Difference output: $S = A \oplus B' \oplus C_{in}$

$AB' \setminus C_{in}$	0	1
00	0	1
01	1	0
11	0	1
10	1	0

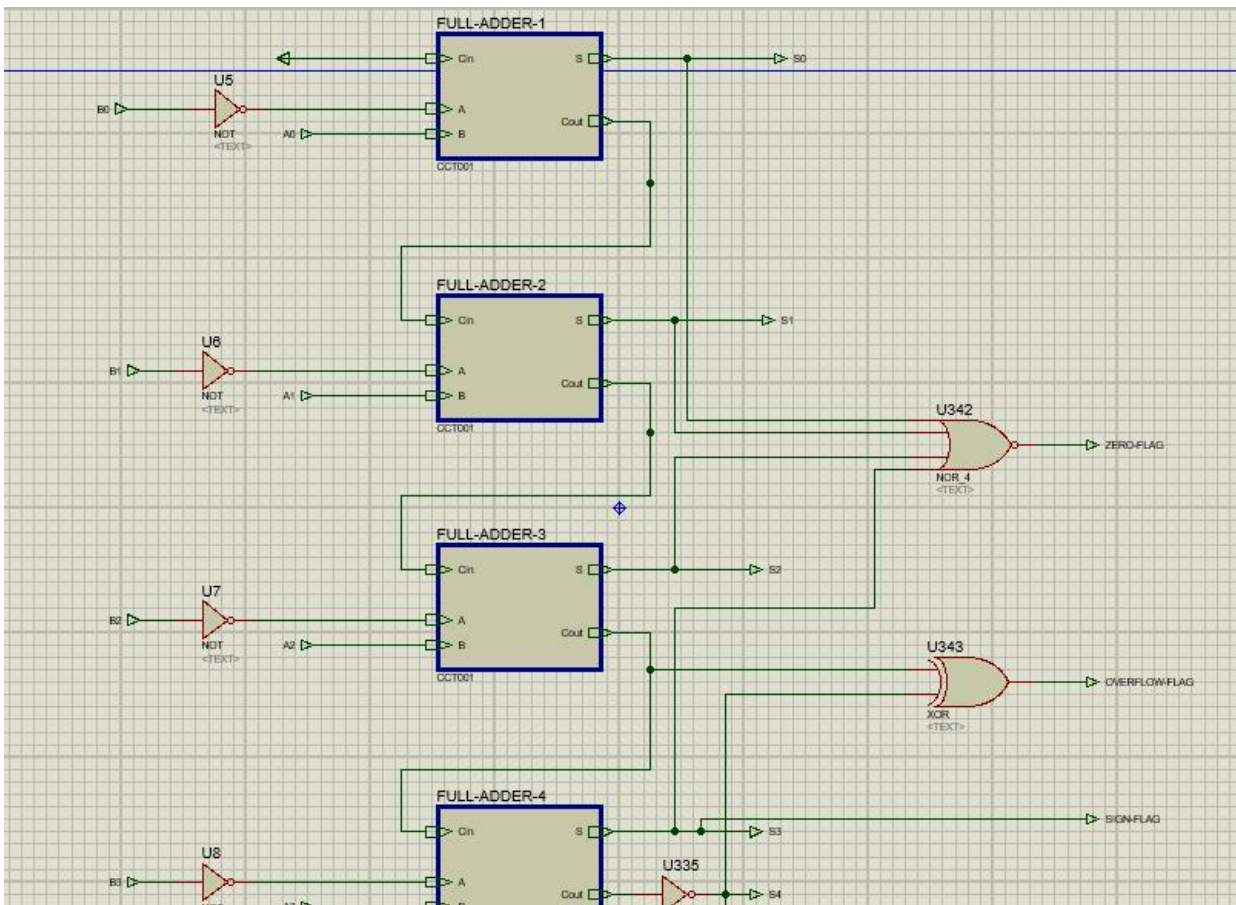
2. Carry Output (Cout) K-Map For the Carry output: $C_{out} = A B' + A C_{in} + B' C_{in}$

$AB' \setminus C_{in}$	0	1
00	0	0
01	0	1
11	1	1
10	0	1

Internal Circuit



Ripple Carry Adder for the Subtractor Circuit



4.3. 4-Bit Comparator Design

A 4-bit magnitude comparator is a combinational logic circuit used to compare two 4-bit binary numbers and determine their relative magnitudes. The comparator produces three outputs indication whether:

- $A > B$
- $A < B$
- $A = B$

Where: $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$

The comparison process begins from the Most Significant Bit (MSB). If the MSBs of both numbers are equal, the circuit proceeds to compare the next lower bits until a difference is detected. The first unequal bit determines which binary number is larger.

Each bit pair is first checked using an XNOR- based equality function and the overall equality output is generated by ANDing all equality signals together

The greater than and less than outputs are generated using hierarchical AND OR logic that prioritizes higher order bits. The implementation uses standard combinational logic gates including XNOR, AND, OR and NOT gates. This structure allows the comparator to be expanded easily for comparing larger binary numbers.

Truth Table Explanation

For 1 – Bit Comparator Truth Table

A	B	A>B	A<B	A=B
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

K-Map Simplification

K-Map simplification for A>B

A/B	0	1
0	0	0
1	1	0

A=1, B=0

Therefore, the simplified Boolean expression is:

$$A>B = AB'$$

K-Map Simplification for A<B

A/B	0	1
0	0	1
1	0	0

A=0, B=1

Therefore, the simplified Boolean expression is:

$$A<B = A'B$$

K-Map Simplification for Equality (A=B)

A/B	0	1
0	1	0
1	0	1

Extension for a 4-Bit Comparator

1, Equality

$$X_3 = A_3'B_3' + A_3B_3$$

$$X_2 = A_2'B_2' + A_2B_2$$

$$X_1 = A_1'B_1' + A_1B_1$$

$$X_0 = A_0'B_0' + A_0B_0$$

$$A=B = X_3X_2X_1X_0$$

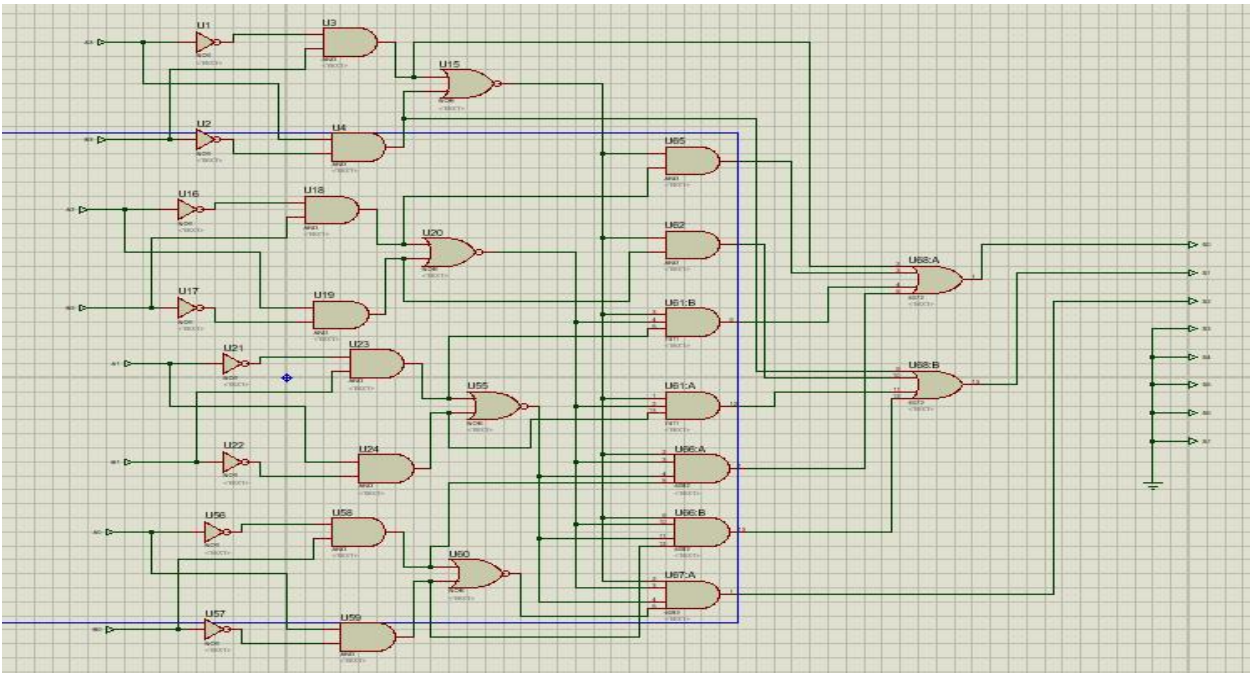
2, Greater Than

$$A>B = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$$

3, Less Than

$$A<B = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$$

Internal Circuit



5. Flags

Flags are special status outputs used in the ALU to indicate the condition of the result after an arithmetic operation. In this project, the ALU uses four flags: Carry Flag, Zero Flag, Sign Flag, and Overflow Flag.

Flag	Function
Carry Flag	Indicates carry generated during arithmetic operation
Zero Flag	Indicates the result is zero
Sign Flag	Indicates the result is negative
Overflow Flag	Indicates arithmetic overflow

6. Operational Selection Mapping

The 4-bit digital calculator utilizes a control unit driven by two selection bits, S1 and S0, to determine which arithmetic or logical operation is performed on the 4-bit inputs, A and B. These selection bits function as the "brain" of the ALU, routing data through the appropriate logic paths—the Ripple Carry Adder (RCA) or the Magnitude Comparator Network—based on the user's input via DIP switches.

6.1. Addition Mode (S1=0, S0=0)

In this configuration, the ALU is set to perform standard binary addition.

- **Logic Path:** The control logic sets the operational mode (M) to 0. This allows the 4-bit inputs A and B to be processed directly by the cascaded full adders without modification.
- **Carry-In Execution:** The initial carry-in (Cin) for the first full adder stage is held at logic 0.
- **Result:** The system outputs a 4-bit binary sum and a carry-out bit to indicate overflow.

6.2. Subtraction Mode (S1=0, S0=1)

Subtraction is implemented using 2's complement arithmetic within the same Ripple Carry Adder structure used for addition.

- **Logic Path:** The selection bit S0=1 sets the mode bit (M) to 1. This activates the XOR-based conditional inversion layer, which bitwise inverts input B ($A + B'$).
- **2's Complement Implementation:** To complete the 2's complement conversion, the S0=1 signal is also fed into the Cin of the first full adder stage. This provides the required "+1" operation, resulting in the calculation: $A + B' + 1$.
- **Result:** The 4-bit output represents the binary difference between the two numbers.

6.3. Comparison Mode (S1=1, S0=0)

When the selection bits are set to S1=1 and S0=0, the arithmetic units are bypassed, and the Magnitude Comparator Network is activated.

- **Logic Path:** Inputs A and B are processed by a dedicated combinational network built from standard logic gate ICs.
- **Flag Generation:** The network evaluates the magnitude of the 4-bit vectors to determine one of three mutually exclusive conditions: $A > B$, $A < B$, or $A = B$.
- **Result:** The active condition is displayed as a high signal on the corresponding output flag (LEDs).

Summary of Selection Mapping

The relationship between the selection bits and the ALU operations is summarized in the following table:

S1	S0	Selected Operation	Logic Path / Mode

0	0	Addition	Adder active; Mode $M=0$
0	1	Subtraction	Adder active; Mode $M=1$ (B inverted, $C_{in}=1$)
1	0	Comparison	Magnitude Comparator Network active

7. Conclusion

In this project, a 4-bit digital calculator (Arithmetic Logic Unit) was successfully designed and analyzed using combinational logic principles. The system was capable of performing three major operations: binary addition, binary subtraction using the 2's complement technique, and magnitude comparison between two 4-bit binary numbers.

The arithmetic section of the design was implemented using a Ripple Carry Adder structure composed of cascaded full adders. Addition was performed directly through the adder circuit, while subtraction was achieved by inverting the second operand and introducing an input carry of one. The comparator section was designed to determine whether one binary number was greater than, less than, or equal to another using Boolean logic and hierarchical bit comparison.

Throughout the project, truth tables, Boolean expressions, and Karnaugh Maps were used to simplify and optimize the logic functions. Standard logic gates including AND, OR, NOT, XOR, and XNOR gates were utilized to implement the complete circuit according to the project requirements.

Simulation and verification confirmed that the designed system operated correctly under different input combinations and operational modes. The project provided practical understanding of combinational circuit analysis, Boolean simplification, arithmetic circuit implementation, and digital system design methodologies.

Overall, the project successfully demonstrated the fundamental concepts of digital logic design and highlighted the importance of ALUs as essential building blocks in modern computing and digital electronic systems.